

A scenic mountain landscape with green hills, rocky cliffs, and a cloudy sky. The foreground shows a rocky, grassy slope. In the middle ground, there are steep, rocky cliffs and green hills. The background features more distant, hazy mountain ranges under a sky filled with white and grey clouds.

AI & Software Engineering Workshop

Week 1, Day 4: Memory and Deploy

Edik Simonian, Summer 2026

The problem

1. Tell your bot: "*my favorite color is blue*"
2. Then ask, in your **very next message**: "*what's my favorite color?*"
3. **It has no idea.**

Each reply forgets the one before it, that's the "stateless mode" notice from Day 1.
It's what we fix today.

Key-value store = lockers

- A wall of lockers 
- **Key** = the locker number → `chat:12345`
- **Value** = whatever you put inside → the conversation
- `get(key)` / `set(key, value)`: that's the whole API
- Ours also has **TTL**: lockers that empty themselves after 30 days

Redis, DynamoDB, memcached, same idea. Ours is SQLite: a database in a single file.

Turn it on

`.env :`

```
SQLITE_PATH=./bot.db
```

- Restart → tell it your favorite color → restart again → ask
- **It remembers.** The file `bot.db` appeared: that's the memory
- No signup, no server, no credit card. It's just a file
- Delete the line → graceful fallback to stateless mode (try it)

How the bot remembers conversations

The model is **stateless**: each API call forgets the last. The memory is *our* code, not the model.

```
# bot/ai.py – runs on every message
history = get_history(user_id)           # load past turns from bot.db
history.append({"role": "user", "content": text})

messages = [{"role": "system", "content": SYSTEM_PROMPT}, *history]
reply = generate(user_id, messages)      # replay the whole chat

history.append({"role": "assistant", "content": reply})
save_history(user_id, history)           # write back: last 20, 30-day TTL
```

Keyed by `chat:<your id>`, the list lives in `bot.db`, so it **survives restarts**.

Live-code: `/remember` and `/recall`

```
@bot.message_handler(commands=["remember"], func=is_allowed)
def cmd_remember(message):
    note = message.text.split(maxsplit=1)[1] if " " in message.text else ""
    store.set(f"note:{message.from_user.id}", note)
    bot.send_message(message.chat.id, "Saved!")
```

`/recall` is the mirror image: `store.get(f"note:{...}").`

Solo build: a full notes feature

- `/remember <text>`: add a note (not replace!)
- `/recall`: list **all** saved notes
- `/forget`: clear them

Hint: the store saves **strings**. To keep a list, `json.dumps` it on the way in, `json.loads` it on the way out.

Production touches already in the repo

- **Rate limit:** `RATE_LIMIT` env var, default 250 messages/user/day
- **History trimming:** last 20 messages per user, 30-day expiry
- **Whitelist:** `ALLOWED_USERS` makes the bot **silent** to strangers
(silence, not rejection: scanners learn nothing)
- **Dedupe:** Telegram retries webhooks; the bot won't answer twice

All built on the same store you just used.

From polling to webhook

- All week: **polling**, your computer asks Telegram for updates
- Production: **webhook**, Telegram pushes to your server's URL
- PythonAnywhere runs the **same Flask app** as an always-on worker
- Same code, same `.env` idea: different delivery

Memory done. Now your bot **stops needing you**.

The front door: `api/index.py`

Route	Job
<code>/api/health</code>	"Is the worker alive?" → <code>OK</code>
<code>/api/webhook</code>	Where Telegram delivers messages
<code>/api/deploy</code>	Pulls new code on push (we'll wire this up)

- The webhook checks a **secret token**: forged requests get 403. The bot generates and registers it itself on first boot.
- `threaded=False` in `bot/clients.py`: handlers must finish in the request, or the platform may kill them mid-reply.

Deploy: one command

1. Sign up at pythonanywhere.com (free Beginner tier)
2. Create an API token: *Account* → *API token*
3. Add to your local `.env`:

```
PA_USERNAME=<your PA username>  
PA_API_TOKEN=<the token>
```

4. Ship it:

```
make deploy-pa
```


What just happened

The script created the web app, cloned **your fork**, built the venv, uploaded a production `.env`, including:

```
SQLITE_PATH=/home/<you>/bot.db  
WEBHOOK_URL=https://<you>.pythonanywhere.com/api/webhook
```

`WEBHOOK_URL` = auto-registration: every boot, the bot tells Telegram "*deliver here.*"
No manual `curl` needed.

Verify like an engineer

1. `https://<you>.pythonanywhere.com/api/health` → `OK`
2. Message your bot: the reply now comes from the server
3. **Quit the bot on your computer (`Ctrl+C`). Message it again.** 🎉
4. Something wrong? *Web tab* → *error log*: read the traceback

Push-to-deploy

One-time setup on your fork, in *Settings* → *Secrets* → *Actions*:

Secret	Value
DEPLOY_SECRET	random string (also in PA <code>.env</code>)
PA_DEPLOY_URL	<code>https://<you>.pythonanywhere.com/api/deploy</code>

Then:

```
git commit -am "New personality" && git push
```

~3 seconds later the live bot is updated. This is how real teams ship.

The fine print

- PA free tier: click the **"Run until 3 months from today"** renewal button **every 3 months**, or the app auto-disables
- PA emails you a week before. Set a phone reminder anyway.
- Free tier limits outbound internet to a whitelist: Telegram and Cerebras are on it

Today → Tomorrow

Today: your bot **remembers**, and it's **live on the internet**, updating on every push.

Tomorrow: the whole day is yours. You design and ship your **own** bot, start to finish.